

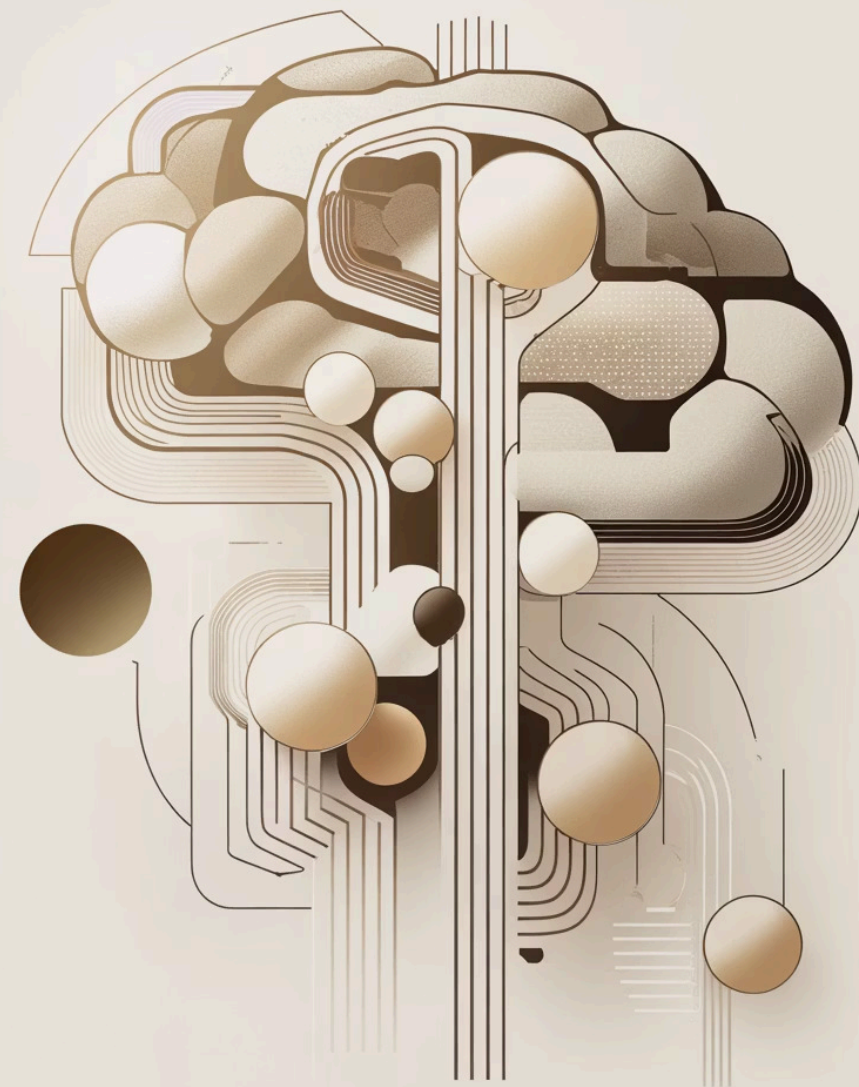
Q-Learning — базовый алгоритм обучения действиям

Обучение агента в среде без модели

Теоретический конспект: фундаментальные основы одного из ключевых алгоритмов обучения с подкреплением

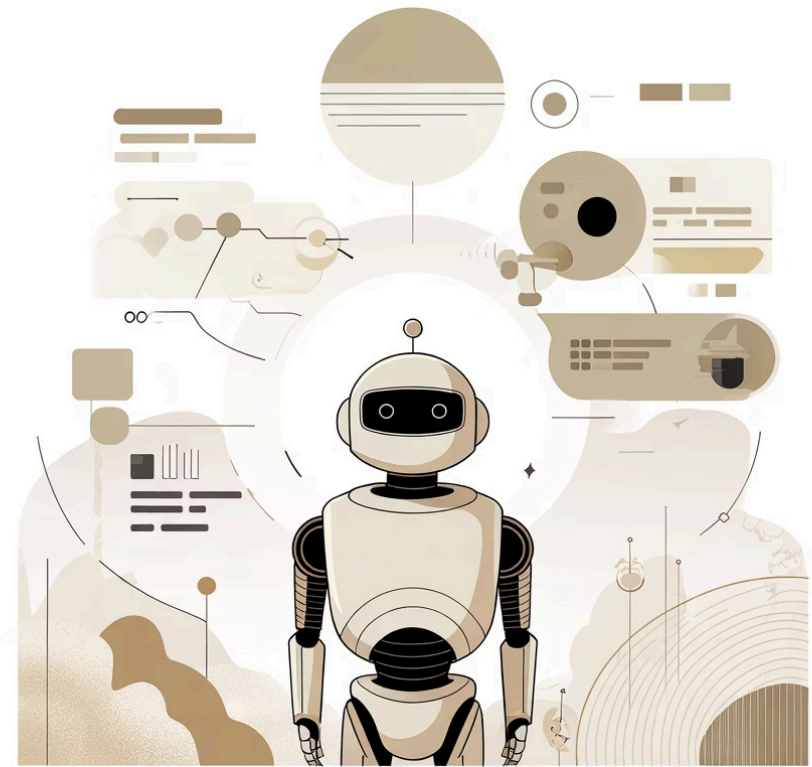
Автор: Саматов Денис

Дата: 06.11.2025



Проблема: как обучать агента без модели среды

- Представьте, что агент (например, робот) оказывается в новой среде. Он не знает её правил и не понимает, какие действия приведут к успеху. Это называется **обучением без модели среды (model-free)**.
- Вместо того чтобы иметь полную "карту" мира, агент учится на **опыте**. Он пробует разные действия, наблюдает за результатами и запоминает, что работает. Со временем он принимает всё более эффективные решения.

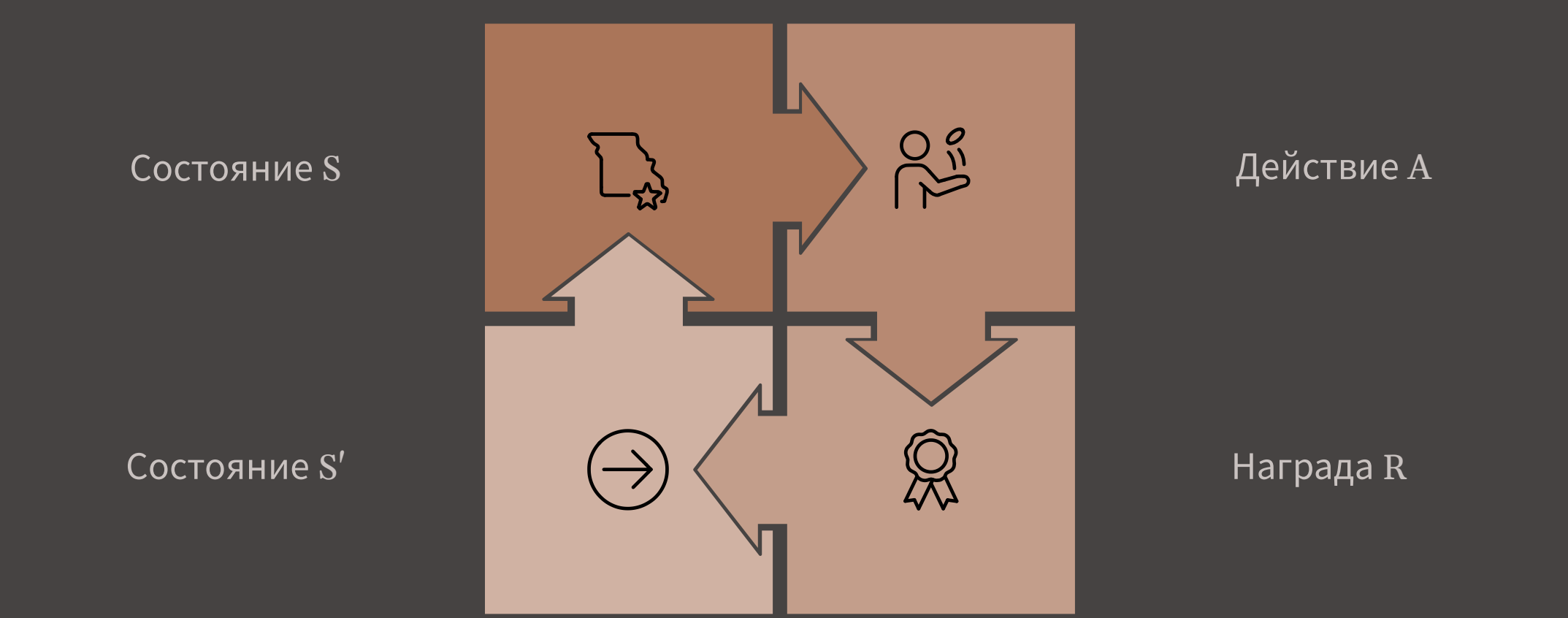


MDP в двух словах

Марковский процесс принятия решений (MDP) — это математическая основа для моделирования принятия решений в ситуа циях, где результаты частично случайны, а частично зависят от лица, принимающего решения. Он описывается набором следующих элементов:

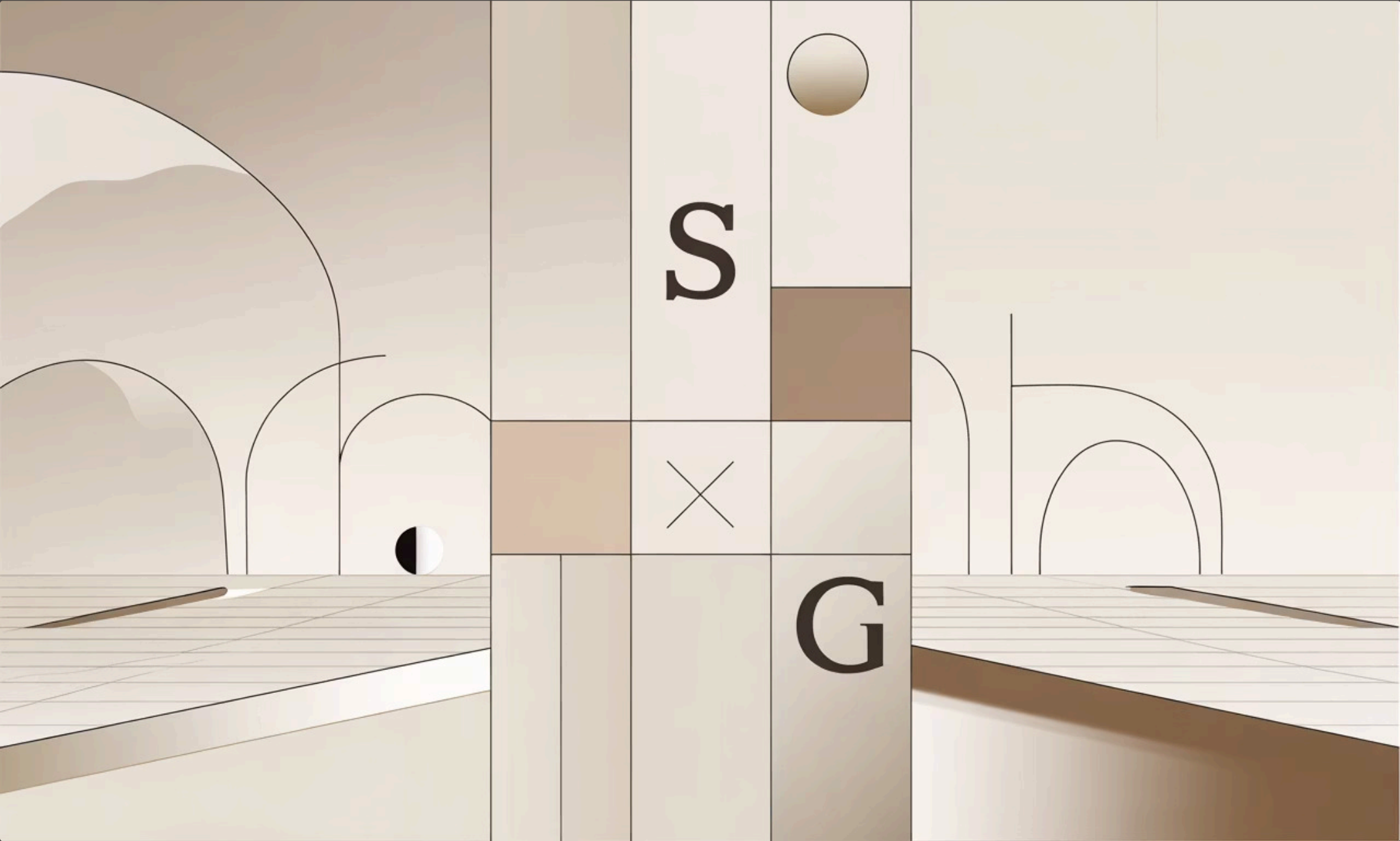
Состояния	s	Текущая ситуация или конфигурация среды, в которой находится агент.
Действия	a	Выбор, сделанный агентом, который влияет на среду.
Награды	r	Числовая обратная связь от среды, указывающая на желательность действия.
Функция переходов	$P(s' s, a)$	Вероятность перехода из состояния s в s' после выполнения действия a .
Политика	$\pi(a s)$	Стратегия агента, определяющая, какое действие a выбрать в данном состоянии s .

Процесс MDP можно представить в виде циклической диаграммы:



Пример с игровой сеткой:

Представьте агента, перемещающегося по сетке. Каждая клетка сетки — это **состояние**. Агент может выполнять **действия**: двигаться вверх, вниз, влево или вправо. Если агент достигает целевой клетки, он получает положительную **награду**. Если попадает в ловушку, получает отрицательную награду. Агент учится выбирать **действия**, чтобы максимизировать свои **награды** со временем.



Off-policy vs On-policy

Off-policy - учится оптимальной политике независимо от того, как действует, On-policy - учится политике, которой фактически следует.

Off-policy

Представьте, что вы учитесь играть в шахматы, наблюдая за игрой двух гроссмейстеров. Вы не играете сами, а просто смотрите и записываете, какие ходы они делают в разных ситуациях, и какие из этих ходов приводят к лучшим результатам. Вы учитесь "оптимальной стратегии", наблюдая, как другие играют, даже если их игра не всегда идеальна.

Это позволяет найти оптимальную политику, даже если агент исследует среду, совершая неоптимальные действия.

On-policy

Теперь представьте, что вы учитесь играть в шахматы, играя сами. Вы делаете ход, видите, что происходит, и корректируете свою стратегию на основе того, что вы реально сделали и что реально произошло после этого. Вы учитесь своей "собственной стратегии", которая может быть неоптимальной на ранних этапах, но улучшается со временем.

Таким образом, алгоритм учится "безопасной" политике, которая учитывает риски исследования, но может не быть истинно оптимальной, если исследовательские действия слишком сильно отклоняются от оптимального пути.



Q-функция и Уравнение Беллмана

Q-функция ($Q(s,a)$)

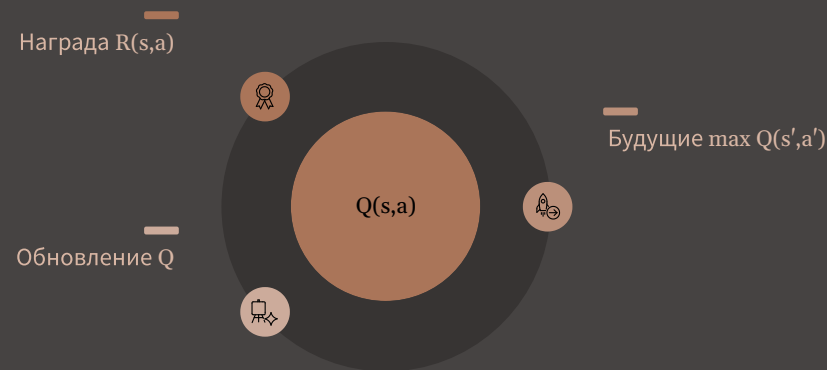
Q-функция ($Q(s,a)$) показывает ожидаемую дисконтированную награду, если совершить действие a в состоянии s и далее следовать оптимальной политике.

Интуиция: «Если я сейчас сделаю действие a в состоянии s , то насколько хорошо это для меня в долгосрочной перспективе?»

Оптимальная Q-функция определяется с помощью уравнения Беллмана:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$$

Это уравнение показывает, как текущая ценность (Q-функция) зависит от немедленной награды и максимальной ожидаемой будущей награды.



Интуиция TD: учимся по шагам

Ключевая идея Temporal Difference (TD) в том, что мы учимся непрерывно, корректируя наши предсказания после каждого действия, не дожидаясь конца эпизода.

Monte Carlo (MC)



Temporal Difference (TD)



- **Учится шаг за шагом:** не нужно ждать конца эпизода.
- **Использует прогноз:** для корректировки текущих ожиданий.
- **Немедленная награда + прогноз будущей ценности.**

TD-ошибка (δ)

$$\delta = r + \gamma Q(s', a') - Q(s, a)$$

TD-ошибка — это разница между тем, что мы ожидали, и тем, что мы на самом деле получили (немедленная награда + дисконтированная ценность следующего состояния).

Общий алгоритм Q-Learning

01

Инициализация

Инициализировать Q-таблицу нулями для всех пар состояние-действие

02

Начало эпизода

Инициализировать начальное состояние s

03

Выбор действия

Выбрать действие a по ϵ -жадной политике (ϵ -greedy)

04

Взаимодействие

Выполнить действие a , получить награду r и новое состояние s'

05

Обновление Q

Применить формулу Беллмана для обновления Q-значения

06

Адаптация

Уменьшить ϵ со временем (меньше случайности $\rightarrow$ больше эксплуатации)

Правило обновления Q-Learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

α

Скорость обучения (Learning Rate): Насколько сильно новые знания влияют на Q-значения (высокое = быстро, низкое = стабильно).

γ

Коэффициент дисконтирования (Discount Factor): Важность будущих наград (0 = только немедленные, 1 = долгосрочные).

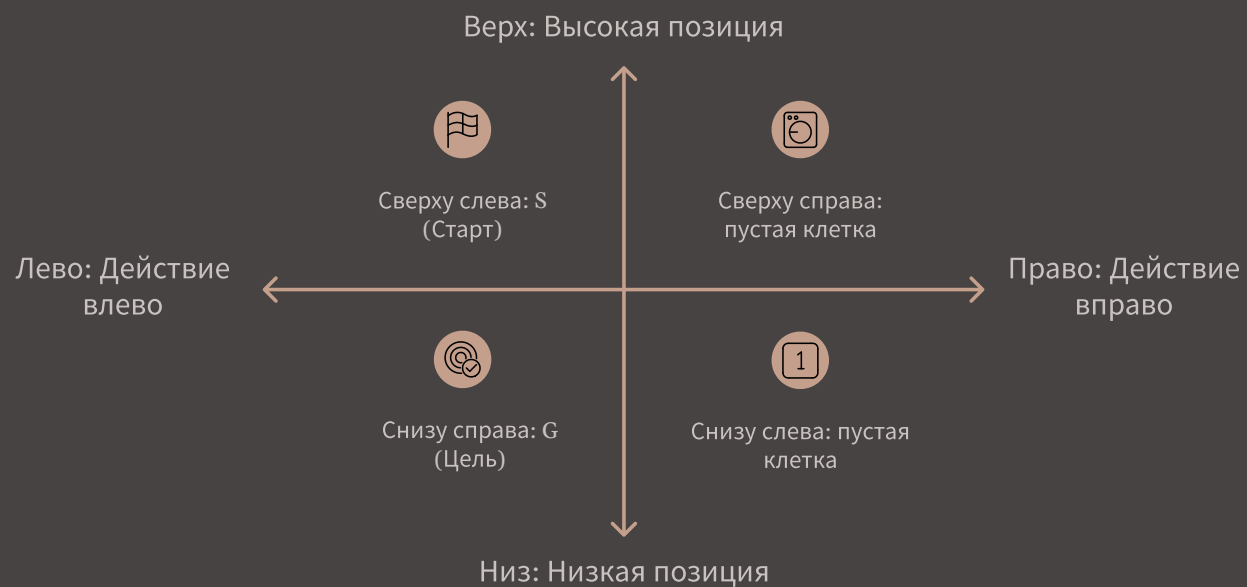
r

Текущая награда (Current Reward): Непосредственная награда за выполненное действие.

$\max_{a'} Q(s', a')$

Лучшее будущее Q-значение: Максимальное Q-значение, которое агент может получить в следующем состоянии.

Обновляем Q-значения по TD-ошибке после каждого шага.



Мини-пример: обучение в сетке 3×3

Рассмотрим простой пример Q-Learning в сетке 3x3, как показано на диаграмме справа. Наш агент начинает в позиции S и должен найти путь до цели G, получая награду +1 при достижении G и 0 за остальные переходы.

Шаг обновления Q-значения

Предположим, агент находится в состоянии s (центральная ячейка) и выполняет действие a (движение вправо), как показано стрелкой на диаграмме. Он переходит в следующее состояние s' (ячейка справа) и получает награду $r = 0$.

Параметры и значения для расчета:

- γ (Коэффициент дисконтирования): 0.9
- α (Скорость обучения): 0.1
- r (Текущая награда): 0
- Текущее Q-значение: $Q(s, a) = 0$
- Максимальное Q-значение в следующем состоянии: $\max_{a'} Q(s', a') = 0.5$

Формула обновления Q-Learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Пошаговый расчет

Давайте подставим наши значения в формулу:

- **Оценка будущего:** $\gamma \max_{a'} Q(s', a') = 0.9 \times 0.5 = 0.45$
- **Целевое Q-значение (Target Q):** $r + \gamma \max_{a'} Q(s', a') = 0 + 0.45 = 0.45$
- **Ошибка предсказания (TD Error):** $[r + \gamma \max_{a'} Q(s', a') - Q(s, a)] = 0.45 - 0 = 0.45$
- **Обновление:** $Q(s, a) \leftarrow 0 + 0.1 \times 0.45 = 0.045$

Таким образом, новое Q-значение для действия "вправо" из центральной ячейки обновляется: $Q(s, a) = 0 \rightarrow 0.045$. Агент постепенно учится ценности этого пути.

Псевдокод Q-Learning

1. Инициализировать Q-таблицу нулями
2. Для каждого эпизода:
3. Инициализировать начальное состояние S
4. Пока S не является терминальным:
5. Выбрать действие A (ϵ -greedy)
6. Выполнить A , получить награду R и S'
7. Обновить $Q(S,A)$ по формуле Q-Learning
8. $S \leftarrow S'$

Off-policy value-based TD-обучение



Практические гиперпараметры

Гиперпараметр	Типичные значения	Влияние
Скорость обучения	0.1-0.5	Определяет, насколько сильно новые знания влияют на текущие Q-значения.
Коэффициент дисконтирования	0.95-0.99	Определяет важность будущих наград по сравнению с немедленными.
Коэффициент исследования	1.0→0.05 (линейно за 1-5 тысяч шагов)	Контролирует баланс между исследованием (exploration) и эксплуатацией (exploitation).
Количество эпизодов	2-5 тысяч (для Gridworld)	Общее количество "игровых сессий" или попыток, за которые агент учится.

Важно отметить, что оптимальные значения этих гиперпараметров сильно зависят от конкретной среды и задачи. Часто требуется экспериментальный подбор (тюнинг).

Q-таблица: память агента

Q-функция представляется в виде **Q-таблицы**, где:

- строки — состояния среды S
- столбцы — возможные действия A
- ячейка $Q(s, a)$ — значение, показывающее «качество» действия a в состоянии s

Пример Q-таблицы:

S0	0.1	0.0	0.5	0.0
S1	0.2	0.8	0.0	0.1
S2	0.0	0.3	0.0	0.9

В этом примере агент считает, что в состоянии S1 наилучшее действие — «вправо» (значение 0.8).

Шаг 1: Инициализация агента

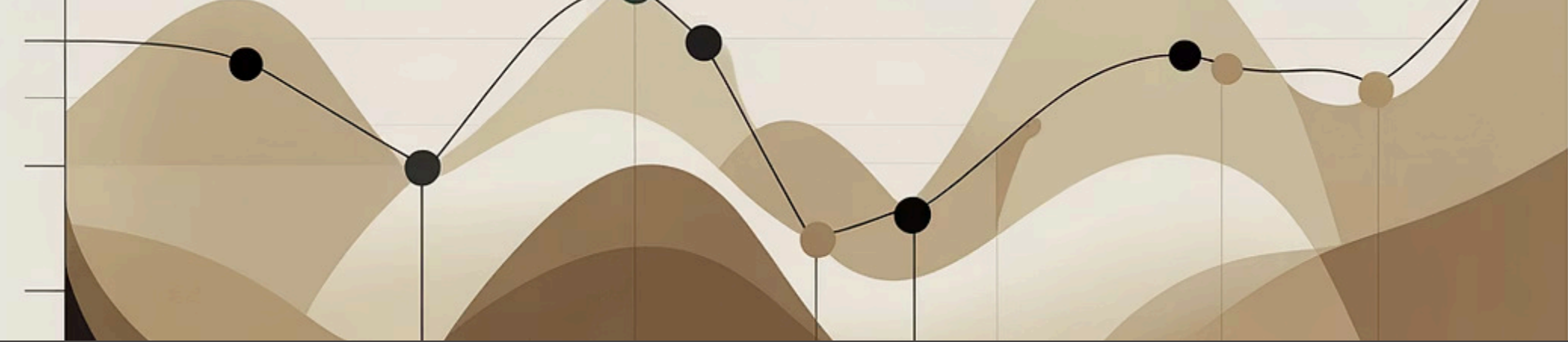
$$Q(s, a) = 0 \text{ для всех } s, a$$

Агент начинает без знаний — «пустая таблица». Все Q-значения устанавливаются в нулевое начальное состояние.

Почему нули? Это нейтральная отправная точка, которая позволяет агенту формировать оценки на основе реального опыта без предвзятости.

Альтернативно, можно использовать случайную инициализацию или оптимистичные начальные значения для стимулирования исследования.





Баланс исследования и эксплуатации: ϵ -greedy

Агент должен найти баланс между двумя вещами: **исследованием** новых возможностей и **эксплуатацией** уже известных лучших действий. Стратегия ϵ -greedy помогает в этом:

$$a_t = \begin{cases} \text{случайное действие,} & \text{с вероятностью } \epsilon \\ \arg \max_a Q(s, a), & \text{с вероятностью } 1 - \epsilon \end{cases}$$

С вероятностью ϵ (эпсилон) агент выбирает **случайное действие**, чтобы исследовать новые пути. С вероятностью $1 - \epsilon$, агент выбирает **лучшее известное действие**, чтобы эксплуатировать уже полученные знания.

Начало обучения

$\epsilon = 1.0$: Агент активно исследует.

Конец обучения

$\epsilon \approx 0.1$: Агент преимущественно эксплуатирует знания.

1

2

3

Середина обучения

$\epsilon = 0.5$: Баланс исследования и эксплуатации.

Важно: значение ϵ постепенно уменьшается со временем. Это позволяет агенту сначала активно исследовать, а затем сосредоточиться на лучших действиях.

Шаг 3: Получение опыта и обновление

Сбор опыта

Агент выполняет действие и получает:

- новое состояние s'
- награду r

Запоминает кортеж опыта:

$$(s_t, a_t, r_{t+1}, s_{t+1})$$



TD-error

Разность между новой оценкой и текущим Q-значением

Шаг 4: Обновление Q-таблицы

Теперь используется **обновление Q-Learning**
(приближение оптимального уравнения Беллмана):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

α (скорость обучения)

Насколько быстро обновляются Q-значения

γ (коэффициент дисконтирования)

Важность будущих наград в оценке текущего состояния

$\max_{a'} Q(s', a')$

Наилучшее возможное будущее Q-значение из следующего состояния

Пример обновления Q-таблицы

Исходные значения:

$$Q(s, a) = 0$$

$$r = 1$$

$$\gamma = 1$$

$$\max_{a'} Q(s', a') = 5$$

$$\alpha = 0.1$$

Расчет обновления:

Используя формулу обновления Q-Learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Подставляем значения:

$$Q(s, a) = 0 + 0.1 \times [1 + 1 \times 5 - 0] = 0.6$$

В результате, значение действия (качество) в этом состоянии выросло до 0.6 — агент "узнал", что это действие является более выгодным.

5. Off-policy

Агент обучается на основе другой политики, чем та, которой он действует.



Действие (Exploration Policy)

Выбор действий происходит по ϵ -**greedy** политике (смешанная политика, включающая случайные действия для исследования).



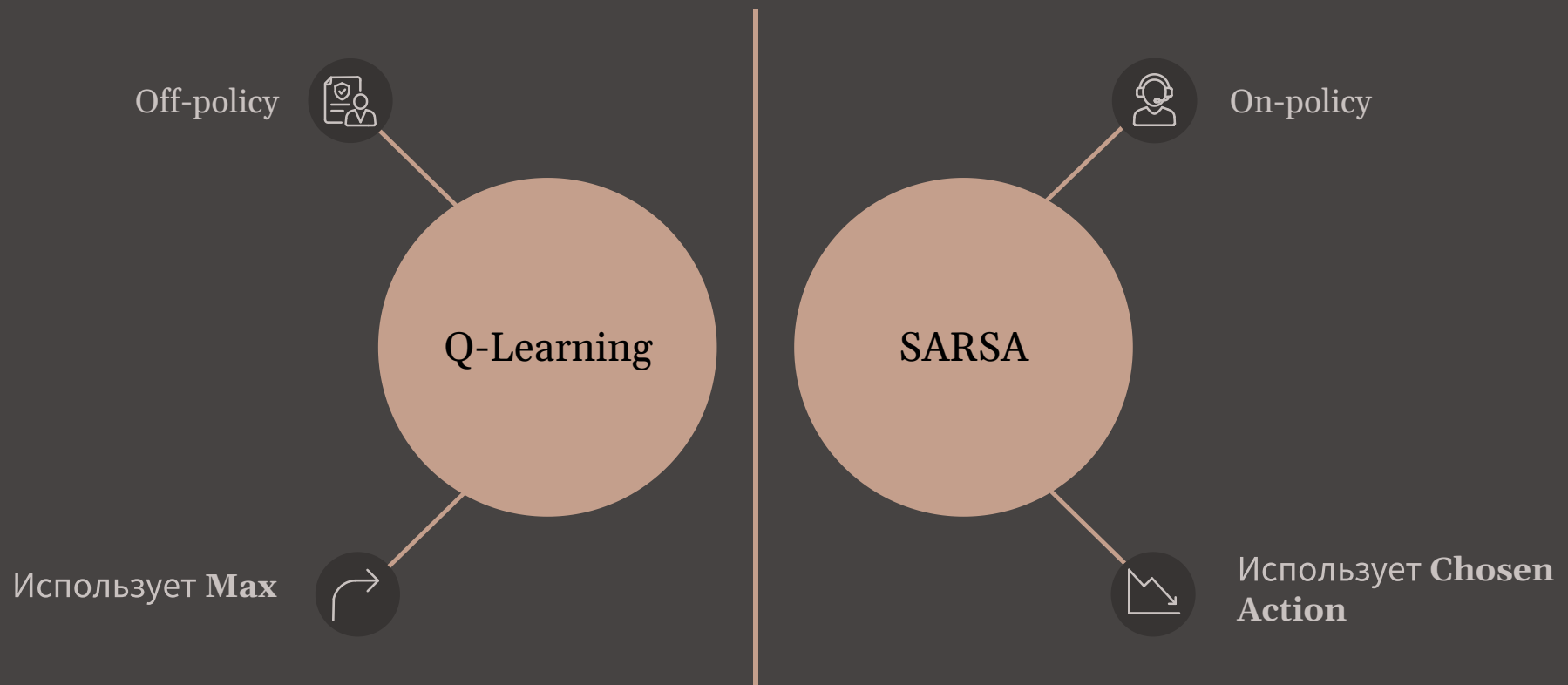
Обновление (Learning Policy)

Обновление Q-значений использует **greedy** политику, основанную на максимальном Q-значении $\max_{a'} Q(s', a')$.

Таким образом:

- Мы *исследуем* мир с ϵ -жадной политикой, чтобы найти новые пути.
- Но *учимся* так, будто всегда выбираем лучшее действие, чтобы оптимизировать будущие решения.

Сравнение: Q-Learning vs SARSA



Q-Learning (Off-policy)

Q-Learning учится оптимальной стратегии независимо от того, какие действия реально предпринимает агент. Для обновления он использует **максимальное Q-значение** для следующего состояния.

$$r + \gamma \max_{a'} Q(s', a')$$

SARSA (On-policy)

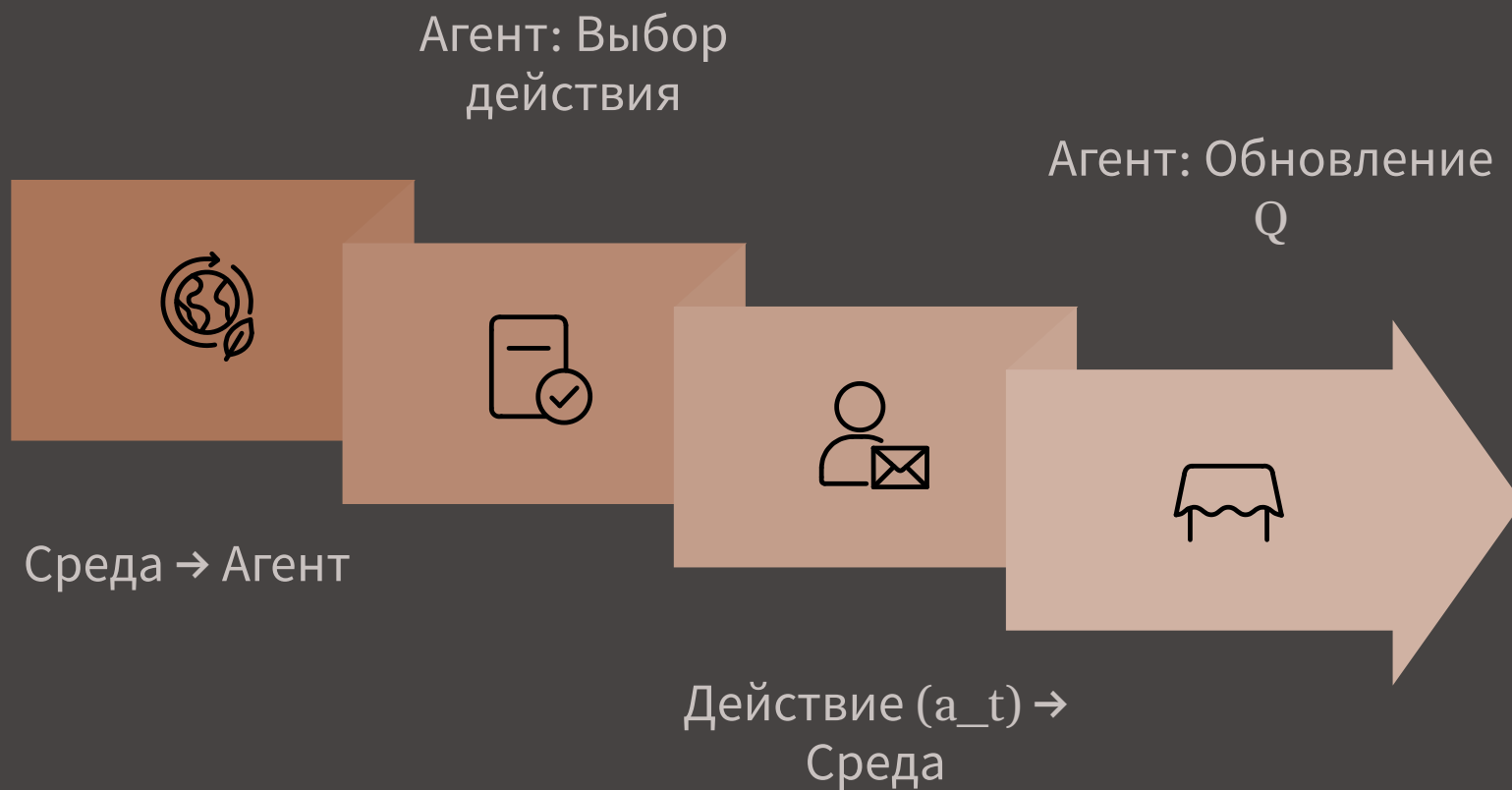
SARSA учится стратегии, которой агент фактически следует (включая исследование). Для обновления он использует Q-значение для **фактически выбранного действия a'** в следующем состоянии.

$$r + \gamma Q(s', a')$$

Таким образом:

- SARSA — безопаснее, так как учитывает исследуемые действия, предотвращая попадание в потенциально опасные состояния.
- Q-Learning — быстрее к оптимуму, поскольку всегда стремится к лучшему возможному будущему, независимо от текущей политики исследования.

7. Итоговая визуальная схема



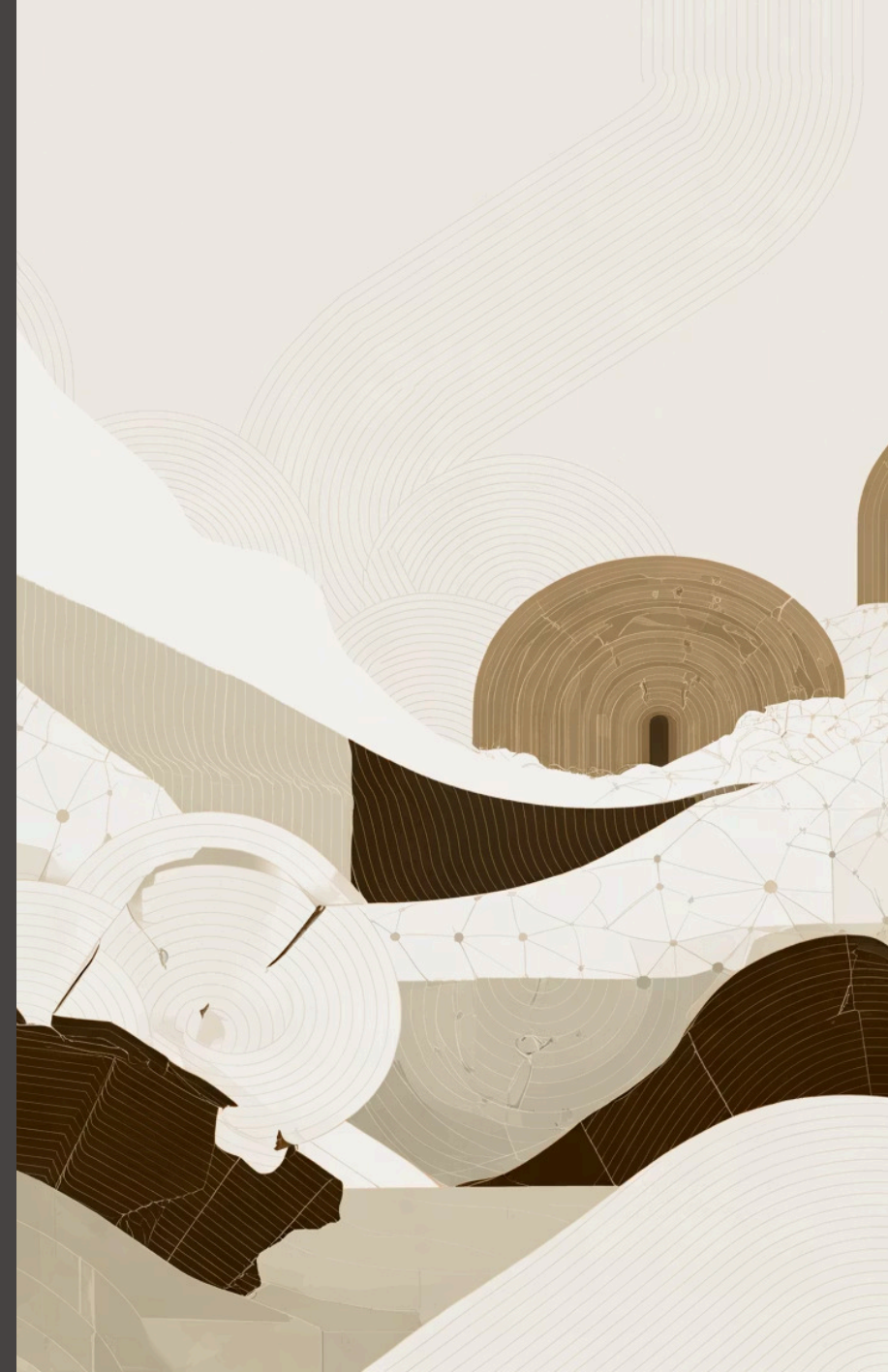
От Q-Learning к DQN: когда Q-таблица не помещается в память

Решение: используем нейросеть-аппроксиматор для масштабирования на сложные задачи

Q-таблица

Аппроксимация

Q-Network



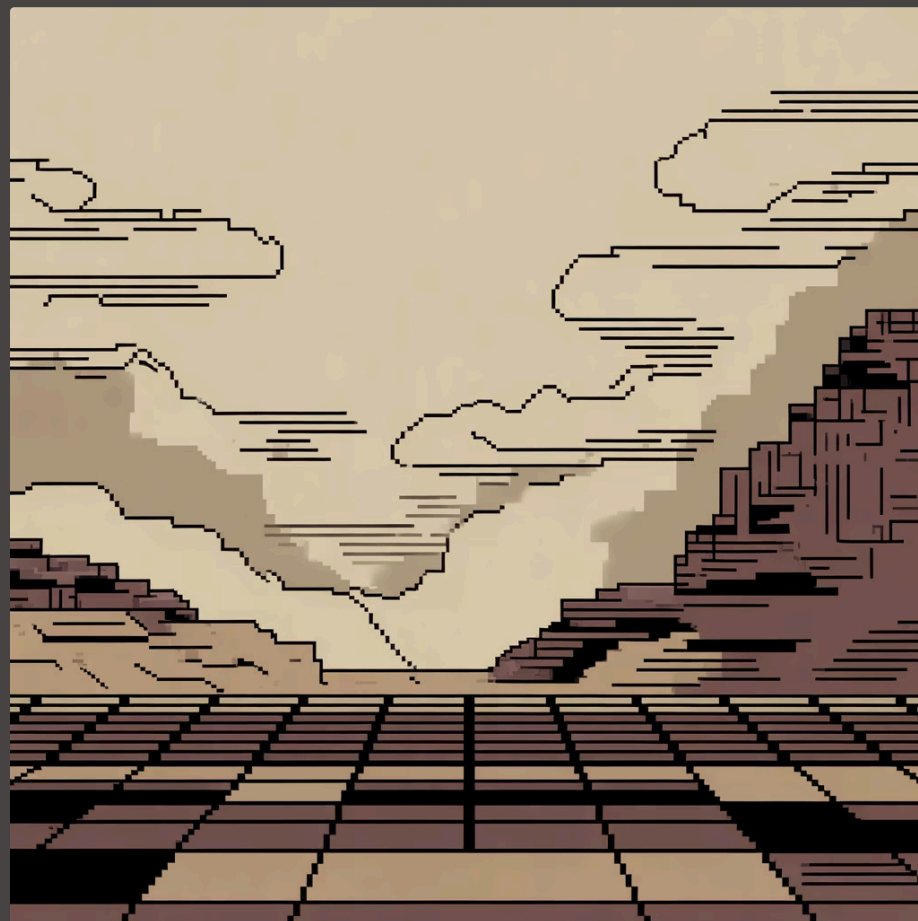
Зачем DQN: когда Q-таблица не масштабируется

Q-обучение на основе таблиц эффективно только для задач с ограниченным и дискретным пространством состояний. В реальном мире многие задачи, однако, обладают огромными или непрерывными пространствами состояний, делая традиционную Q-таблицу непрактичной.

Рассмотрим пример Atari-игр, где каждый кадр (210×160 пикселей, 3 цветовых канала, каждый с 256 значениями) представляет собой состояние. Количество возможных состояний составит:

$$256^{210 \times 160 \times 3} = 256^{100800}$$

Это число значительно превышает количество атомов во Вселенной. Построение и заполнение Q-таблицы для такого огромного пространства состояний становится абсолютно невозможным.



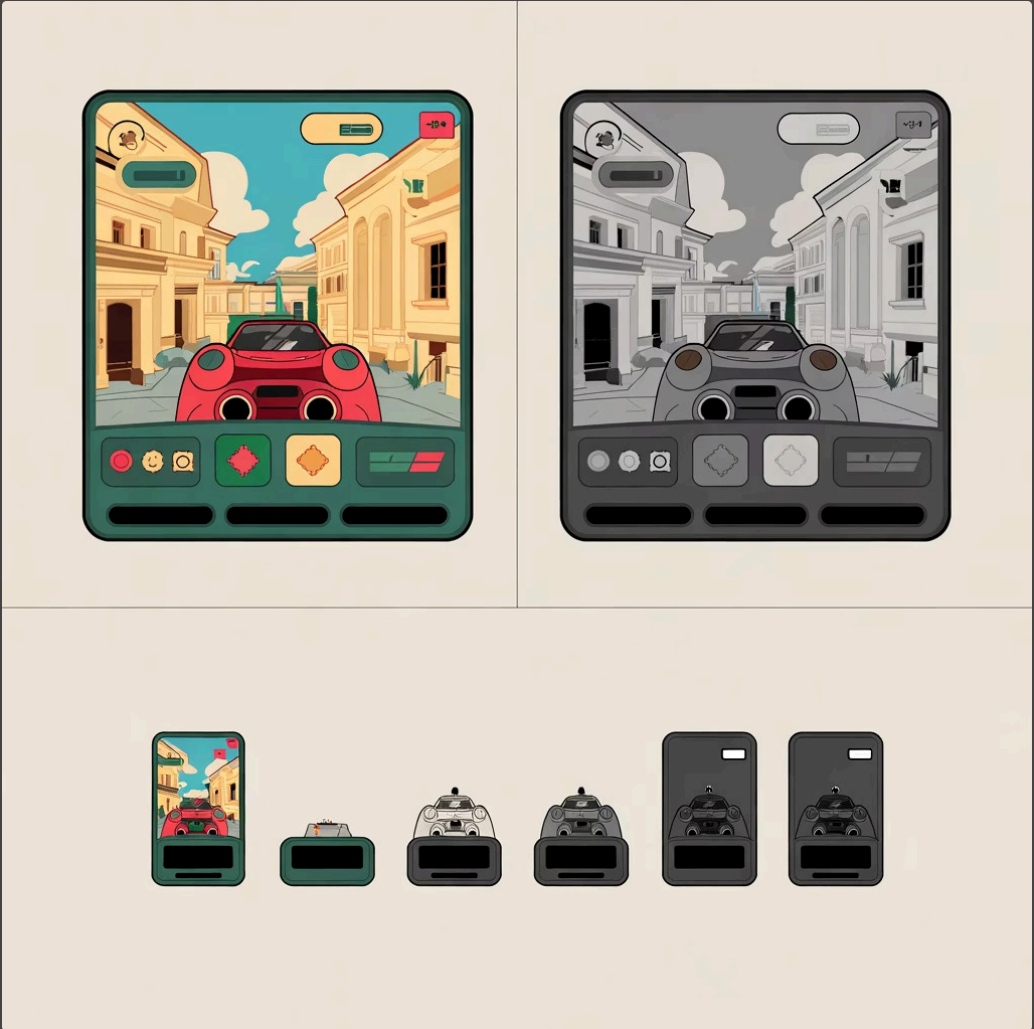
DQN: нейросеть вместо таблицы

Для масштабирования Q-Learning на сложные среды с высокоразмерными или непрерывными пространствами состояний требуется иной подход. Deep Q-Networks (DQN) используют нейронные сети для аппроксимации Q-функции, а не Q-таблицы.

Препроцессинг входных данных

Чтобы эффективно обрабатывать входные данные, особенно из визуальных сред, применяется серия шагов препроцессинга:

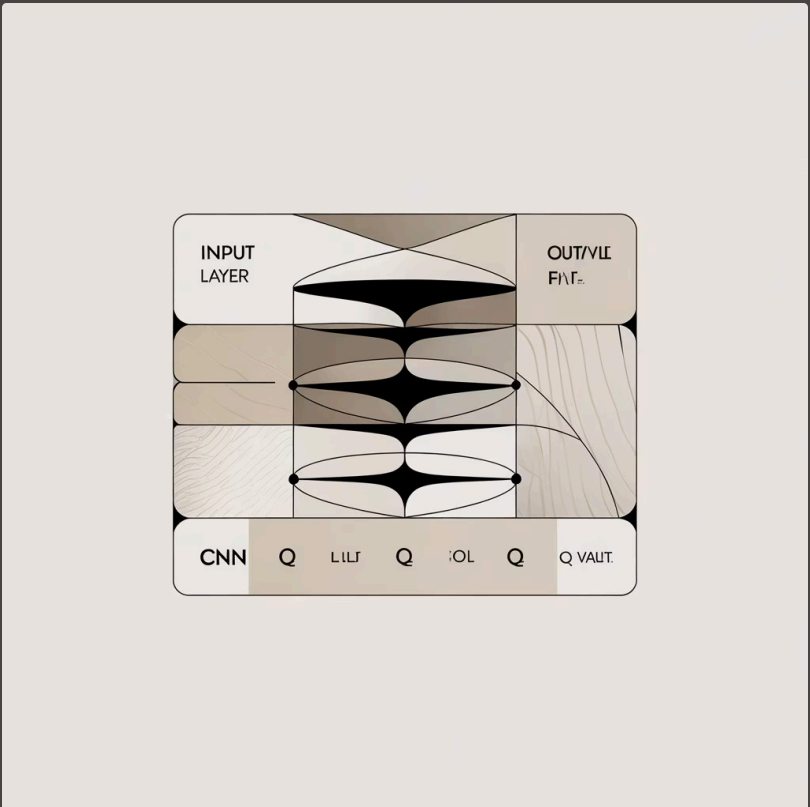
- **Перевод в оттенки серого:** уменьшает размерность.
- **Масштабирование:** до фиксированного размера (84×84 пикселей).
- **Стек из 4 кадров:** для учёта динамики.



Архитектура нейронной сети (Nature DQN)

Оригинальная архитектура Nature DQN использует свёрточные нейронные сети для извлечения признаков из визуальных входных данных:

- 3 свёрточных слоя (CNN).
- Полностью связный слой (FC).
- Выходной слой: Q-значения для всех возможных действий.



Два стабилизатора DQN

Для стабилизации и повышения эффективности обучения в DQN используются две ключевые техники:



Replay Buffer

Хранит опыт агента (переходы). Обучение на перемешанных батчах разрывает корреляции между последовательными образцами, стабилизируя обучение.



Target Network

Копия параметров онлайн-сети. Обновляется редко, периодически копируя параметры онлайн-сети. Стабилизирует целевые Q-значения, предотвращая постоянное изменение цели.

Улучшения DQN

Для дальнейшего повышения эффективности обучения DQN были предложены следующие улучшения:



Double DQN

Борется с переоценкой Q-значений.



Dueling Network

Разделяет ценность состояния ($V(s)$) и преимущество действия ($A(s,a)$).



Prioritized Experience Replay (PER)

Выбирает важные переходы для обучения.

Алгоритм обучения DQN

Цикл обучения Deep Q-Network включает несколько ключевых шагов, обеспечивающих стабильность и эффективность:

		
Инициализация Инициализация параметров θ (для онлайн-сети) и θ^- (для целевой сети), а также пустого буфера воспроизведения D.	Выбор действия Агент выбирает действие a_t в состоянии s_t с использованием ϵ-жадной стратегии (исследование/эксплуатация).	Выполнение действия Выбранное действие a_t выполняется в среде, наблюдается переход к новому состоянию s_{t+1}, награда r_t и флаг <code>done_t</code> (окончание эпизода).
		
Добавление в буфер Переход $(s_t, a_t, r_t, s_{t+1}, \text{done}_t)$ сохраняется в буфере воспроизведения D.	Выборка минибатча Из буфера D случайным образом выбирается минибатч переходов.	Расчет целевых Q-значений Для каждого перехода в минибатче рассчитывается целевое Q-значение y_j с использованием целевой сети θ^- (для стабилизации): $y_j = r_j + \gamma \max_{a'} Q_{\theta^-}(s_{j+1}, a')$ если не done_j, иначе r_j
		
Оптимизация онлайн-сети Параметры онлайн-сети θ обновляются путём минимизации функции потерь $L(\theta)$ между предсказанными Q-значениями и целевыми y_j: $L(\theta) = \mathbb{E}_{(s,a,r,s',\text{done}) \sim D} [(y - Q_{\theta}(s, a))^2]$	Обновление целевой сети Параметры целевой сети θ^- периодически обновляются (например, каждые C шагов) путём копирования параметров онлайн-сети: $\theta^- \leftarrow \theta$.	Обновление ϵ Значение ϵ для ϵ-жадной стратегии постепенно уменьшается со временем (decay) для перехода от исследования к эксплуатации.

Гиперпараметры и метрики DQN

Типичные гиперпараметры для DQN, особенно в среде Atari, играют ключевую роль в стабильности и производительности обучения:

Коэффициент дисконтирования (γ)

$\gamma = 0.99$: Определяет важность будущих наград. Высокое значение указывает на дальновидное планирование.

Размер мини-батча

$batch = 32$: Количество образцов, выбираемых из буфера воспроизведения для одного шага градиентного спуска.

Размер буфера воспроизведения

$|D| = 10^6$: Максимальное количество переходов (s, a, r, s') , которое хранится в буфере для обучения.

Скорость обучения (α_{Adam})

$\alpha_{Adam} \approx 2.5 \times 10^{-4}$: Скорость обучения для оптимизатора Adam, влияющая на размер шага при обновлении весов нейронной сети.

Частота обновления целевой сети (C)

$C \in [10^3, 10^4]$: Количество шагов, после которых параметры онлайн-сети копируются в целевую сеть.

Стратегия ϵ -жадного выбора

$\epsilon : 1.0 \rightarrow 0.1$ за 10^6 шагов: Начальное значение ϵ для исследования, которое постепенно уменьшается, чтобы агент переходил к эксплуатации.

Для оценки эффективности обучения DQN используются следующие ключевые метрики:

→ Средняя награда эпизода	→ Доля действий с максимальным Q	→ Устойчивость лосса
Сумма наград, полученных агентом за один эпизод, усредненная по нескольким эпизодам. Является прямым показателем производительности агента.	Показывает, как часто агент выбирает действия, которые соответствуют максимальному Q-значению, предсказанному сетью. Отражает уверенность агента и переход от исследования к эксплуатации.	Изменение функции потерь во время обучения. Стабильное снижение лосса указывает на успешную сходимость нейронной сети.

Резюме: от Q-Learning к DQN

Deep Q-Networks (DQN) значительно расширили возможности классического Q-Learning, преодолев его ограничения и открыв путь к решению сложных задач обучения с подкреплением.

Представление Q-функции	Таблица Q-значений	Глубокая нейронная сеть
Масштабируемость	Ограничена небольшими пространствами состояний	Высокая, для высокоразмерных состояний (например, пиксели)
Стабильность обучения	Может быть нестабильным в сложных средах	Улучшена за счет буфера опыта и целевой сети

Классический Q-Learning

Классический Q-Learning — это алгоритм обучения без модели, который учится находить оптимальную политику, напрямую оценивая функцию ценности действия-состояния $Q(s,a)$. Он использует итеративное обновление для приближения оптимальной Q-функции:

$$Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a')]$$

Этот метод хорошо работает в средах с небольшим числом состояний, но не масштабируется для сложных, высокоразмерных задач.

Deep Q-Network (DQN)

DQN расширяет Q-Learning, используя нейронные сети для аппроксимации Q-функции, что позволяет обрабатывать высокоразмерные пространства состояний и действий. Ключевые идеи DQN включают:

- Нейронная сеть вместо таблицы Q-значений.
- Буфер воспроизведения опыта для разрыва корреляций.
- Целевая сеть для стабилизации обучения.

TD-цель DQN и функция потерь:

$$y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$$

$$L(\theta) = \mathbb{E}[(y - Q_{\theta}(s, a))^2]$$

DQN обеспечивает преимущества перед классическим Q-Learning, такие как:



Масштабируемость

Эффективная работа с огромными пространствами состояний благодаря глубоким нейронным сетям.



Стабильность

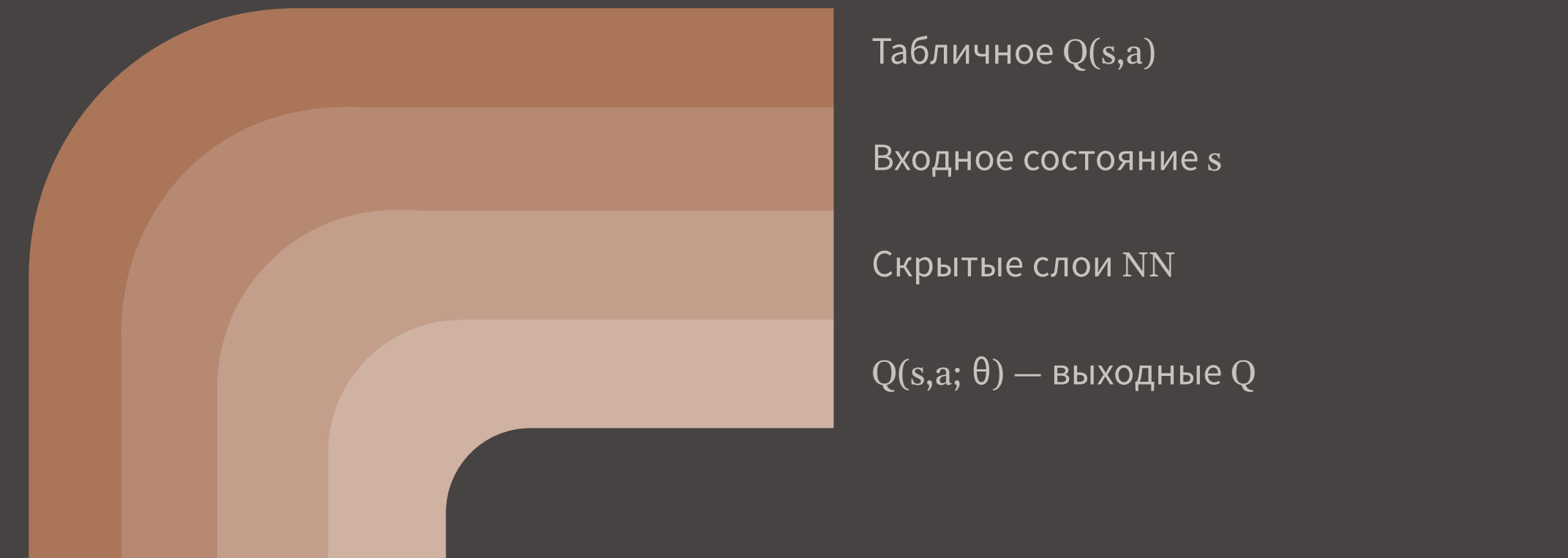
Более стабильное обучение за счет буфера воспроизведения опыта и целевой сети.



Эффективность

Достижение человеческого уровня производительности в сложных средах, таких как игры Atari.

Переход от табличного Q-Learning к нейросетевому DQN



Итоги и когда что выбирать



Q-Learning: Простота vs. Масштабируемость

Q-Learning прост и прозрачен, но плохо масштабируется на большие пространства состояний и действий.



Важность ϵ -распада и исследования

Для эффективного обучения критически важен правильный распад коэффициента исследования ϵ и адекватный охват пространства состояний.



DQN: Решение масштабирования

Deep Q-Networks (DQN) успешно решают проблему масштабирования Q-Learning для больших дискретных пространств, но требуют техник стабилизации (например, буфер воспроизведения опыта, целевые сети).



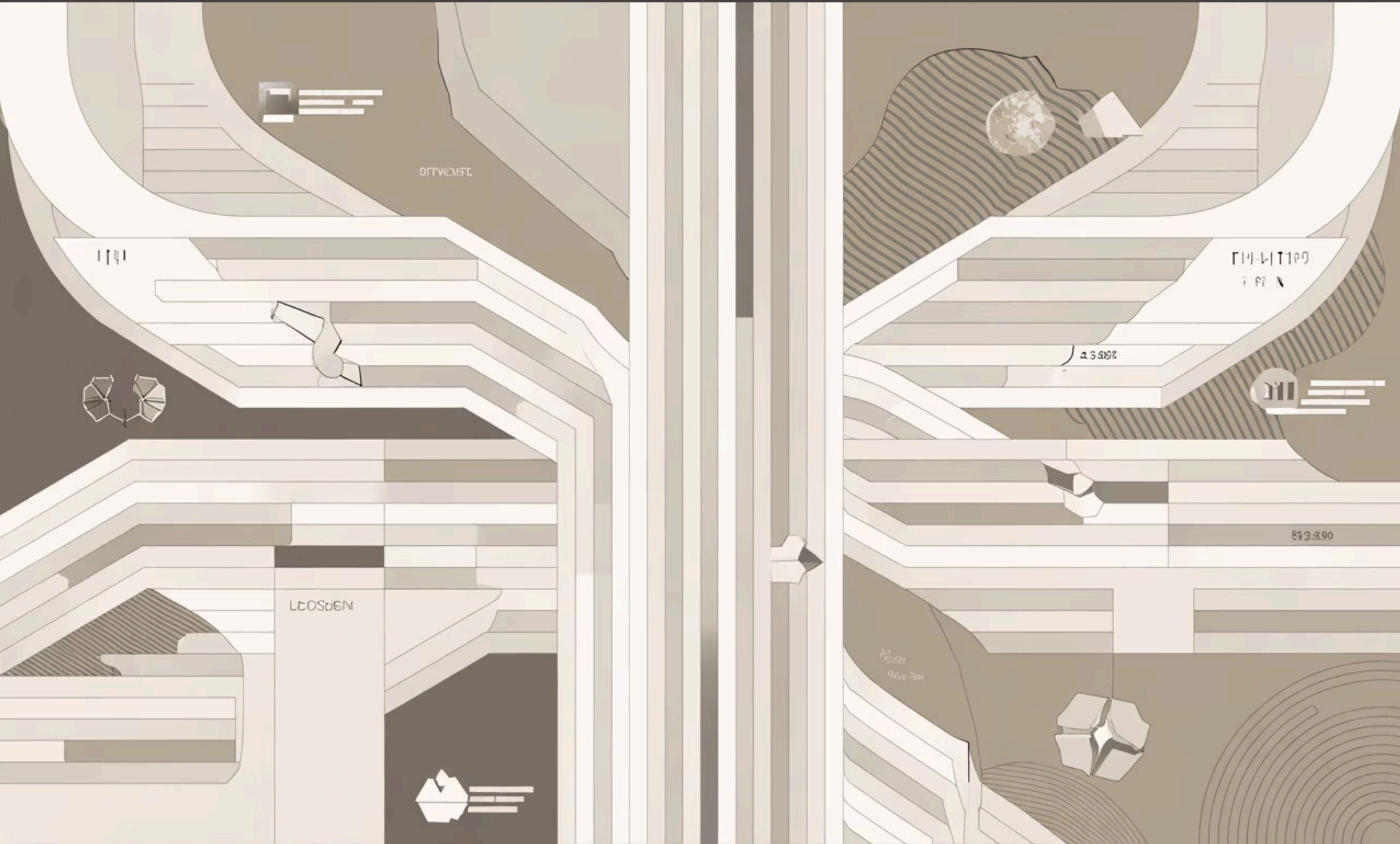
Непрерывные пространства действий

В задачах с непрерывными пространствами действий Q-Learning и DQN неэффективны; здесь применяются другие методы, такие как DDPG (Deep Deterministic Policy Gradient) или SAC (Soft Actor-Critic).



Критерий выбора метода

Выбор алгоритма сильно зависит от характеристик среды: размера пространства состояний/действий, их дискретности/непрерывности.



Понимание этих принципов позволяет выбирать наиболее подходящий алгоритм для конкретной задачи обучения с подкреплением.